

HANDS-ON ENGINEERING DEEP-DIVE

AWS Infrastructure Troubleshooting & Automation

A real-world breakdown of diagnosing, fixing, and automating complex cloud infrastructure failures across AWS, Terraform, and Python/Boto3.

AWS Cloud

Terraform IaC

Python / Boto3

pytest

CORE ENGINEERING WORKFLOW

`"I built it → it failed → I investigated the AWS layer → found the root cause → fixed it → automated/validated it."`

SK ASIF RAHEMAN

MCA Graduate | AWS Cloud & Infrastructure

FOCUS

AWS • IaC • Boto3

REAL AWS TROUBLESHOOTING

Deep-dive into live runtime errors, IAM permission barriers, and S3 policy resolution.

01 Lambda → DynamoDB AccessDeniedException

Internal Server Error

SYMPTOM	API returned <code>HTTP 500 Internal Server Error</code> on request execution.
INVESTIGATION	Checked Lambda execution role → IAM policy permissions → DynamoDB target resource bindings.
ROOT CAUSE	The Lambda configuration and Terraform-managed DynamoDB table name were inconsistent.
RESOLUTION	Passed the Terraform-managed table name directly through the <code>TABLE_NAME</code> environment variable.
RESULT	<code>HTTP 200 OK</code> returned with expected DynamoDB response data.

02 S3 Frontend 403 Access Denied

HTTP 403 Forbidden

SYMPTOM	Frontend web client could not be accessed by end users.
INVESTIGATION	Checked S3 public-access configuration flags and verified bucket policy statements.
ROOT CAUSE	The required public access configuration did not match the underlying bucket policy rules.
RESOLUTION	Corrected the S3 public access settings and synchronized the Terraform-managed policy.

TERRAFORM / DEPLOYMENT TROUBLESHOOTING

Resolving initialization locks and dynamic dependency mismatches in automated deployments.

03 Terraform Provider Error

Provider Unavailable

ERROR	Required AWS provider plugin was unavailable or missing from local workspace cache.
INVESTIGATION	Checked Terraform initialization state and plugin directory structure.
RESOLUTION	Ran <code>terraform init</code> to fetch registry plugins and verified provider installation integrity.

04 Frontend Using Old API Endpoint

Stale Endpoint Call

SYMPTOM	Frontend continued calling an outdated API endpoint URL after complete infrastructure recreation.
ROOT CAUSE	API Gateway received a completely new API ID upon redeployment, invalidating static frontend references.
RESOLUTION	Dynamically generated the frontend JavaScript endpoint configuration directly through Terraform outputs instead of hardcoding static URLs.

PYTHON + BOTO3 INFRASTRUCTURE INSPECTION

Custom programmatic audit layer built to replace slow manual console checks across 7 AWS services.

Automation Strategy: Instead of manually navigating and checking every individual AWS service through the console, I built a Python inspection script.

• Lambda

- Functions
- Configuration
- Pagination

• DynamoDB

- Tables
- Configuration
- Items

• S3

- Buckets
- Objects
- Object retrieval

• API Gateway

- APIs
- Routes
- Stages

• CloudWatch

- Log groups
- Log streams
- Log events

• CloudTrail

- Trails
- Configuration
- Recent events

• IAM

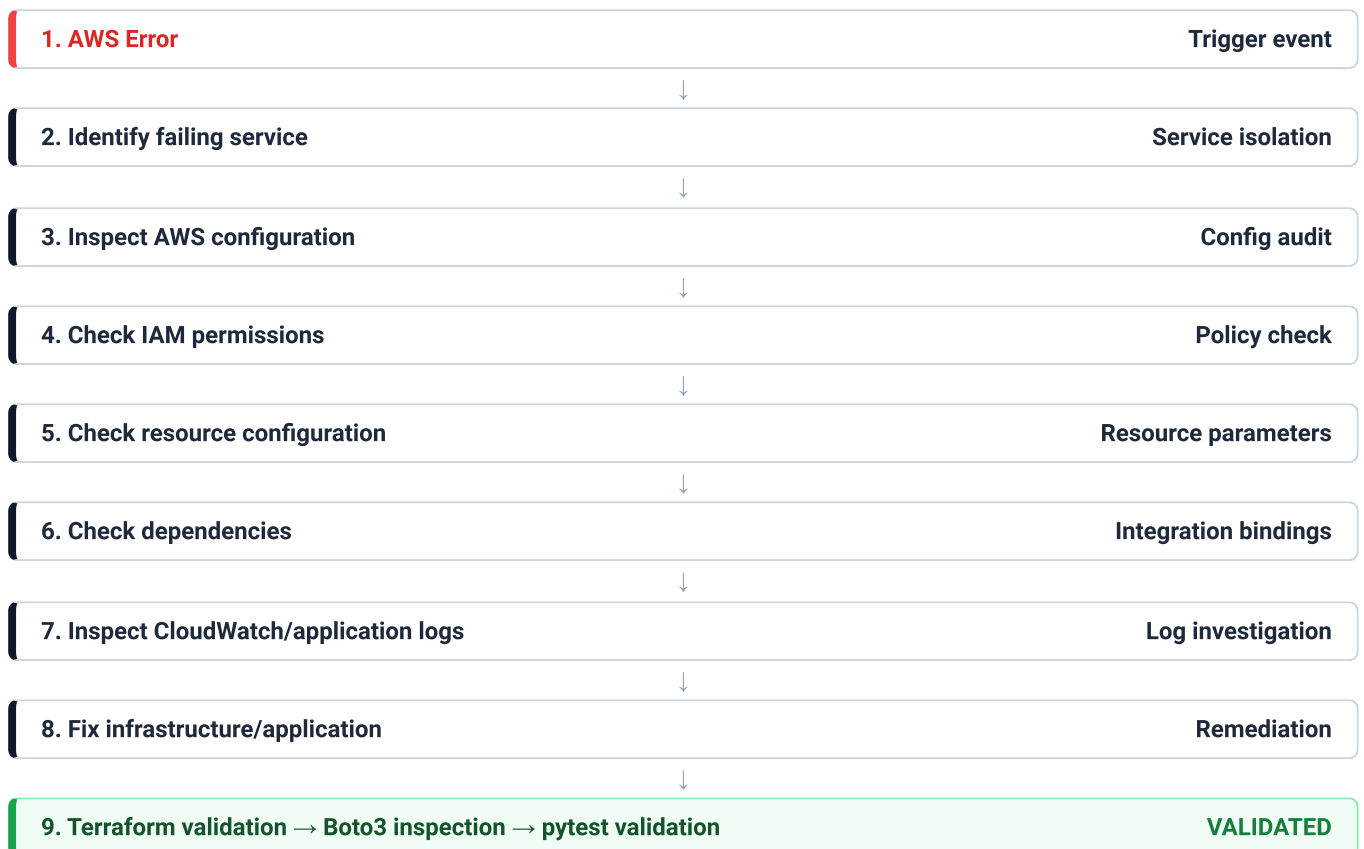
- Roles
- Inline policies
- Policy documents

ADDITIONAL HANDLING

- AWS API pagination
- ClientError handling
- Resource filtering
- Structured API response inspection

FROM FAILURE TO VALIDATION

Systematic engineering lifecycle used to diagnose, rectify, and continuously validate cloud resources.



VALIDATION

pytest validates:

• API Gateway

• DynamoDB

• Lambda

• S3

TERRAFORM WORKFLOW

terraform fmt



terraform validate



terraform plan



terraform apply